De duplication Using Locality Sensitive Hashing

Introduction

One of the most fundamental problem of data mining is to find the **similar** item. De duplication is one example of finding similar items.
Applications of finding similar items:

1. Filter duplicate documents in search engine
2. Plagiarism, mirror pages
3. Recommend similar products, movies.

Problem Statement

The aim of this mini project is to implement de duplication model to find the duplicate documents from the large collection of documents via clustering using locality sensitive hashing and prune the duplicate documents.

Background [1]

Jaccard Similarity of Sets

Jaccard similarity is measure of similarity between two sets. It is ratio of the intersection to the union of two sets.
Let S_1 and S_2 are two sets, then the jaccard similarity between is defined as

$$Sim(S_1, S_2) = \frac{|S_1 \cap S_2|}{|S_1 \cup S_2|}$$

Document to set

To find the lexically similar document, the most effective way is to construct the set corresponding to each document of the sub-string that appear in the document. The common method to model text as set are **Bag of Word Model** and **Shingle**. A Bag of Word model is an unordered collection of word, which keeps the multiplicity of the words. A more general approach is to shingle the document. This takes consecutive words and group them as a single object.

k-Shingle

A k-shingle is consecutive sub-string of k-words(or k-characters) of the document string. The alternate name for k-shingle is k-gram.

Jaccard similarity With Shingles

Let
D1: De duplication Using Locality Sensitive Hashing
D2: Using Locality Sensitive Hashing for De duplication.

The 2-shingle(word level) for document D1 and D2 are as follow:
D1 = {“De duplication”, “duplication Using”, “Using Locality”, “Locality Sensitive”, “Sensitive Hashing”}
D2 = {“Using Locality”, “Locality Sensitive”, “Sensitive Hashing”, “Hashing for”, “for De”, “De duplication”}
D1 ∩ **D2** = {“De duplication’, “Using Locality”, “Locality Sensitive”, “Sensitive Hashing”}
D1∪**D2** = {“De duplication”, “duplication Using”, “Using Locality”, “Locality Sensitive”, “Sensitive Hashing”, “Hashing for”, “for De”}

$$Sim(D1, D2) = \frac{|D1 \cap D2|}{|D1 \cup D2|} = \frac{4}{7} = 0.57$$

The 2-shingle(character level) for document D1 and D2 are as follow:
D1 = {“De”, “e ”, “ d”, “du”, “up”, “pl”, “li”, “ic”, “ca”, “ai”, “it”, “to”, “on”, “n ”, “ U”, “Us”, “si”, “in”, “ng”, “g ”, “ L”, “Lo”, “oc”, “al”, “ty”, “y ”, “ S”, “Se”, “en”, “ns”, “ti”, “iv”, “ve”, “ H”, “Ha”, “as”, “sh”, “hi”}
D2 = {“Us”, “si”, “in”, “ng”, “g ”, “ L”, “Lo”, “oc”, “ca”, “al”, “li”, “it”, “ty”, “y ”, “ S”, “Se”, “en”, “ns”, “ti”, “iv”, “ve”, “e ”, “ H”, “Ha”, “as”, “sh”, “hi”, “ f”, “fo”, “or”, “r ”, “ D”, “De”, “ d”, “du”, “up”, “pl”, “ic”, “at”, “io”, “on” }
D1 ∩ **D2** = {“De”, “e ”, “ d”, “du”, “up”, “pl”, “li”, “ic”, “ca”, “it”, “on”, “Us”, “si”, “in”, “ng”, “g ”, “ L”, “Lo”, “oc”, “al”, “ty”, “y ”, “ S”, “Se”, “en”, “ns”, “ti”, “iv”, “ve”, “ H”, “Ha”, “as”, “sh”, “hi”}
D1∪**D2** = {“De”, “e ”, “ d”, “du”, “up”, “pl”, “li”, “ic”, “ca”, “ai”, “it”, “to”, “on”, “n ”, “ U”, “Us”, “si”, “in”, “ng”, “g ”, “ L”, “Lo”, “oc”, “al”, “ty”, “y ”, “ S”, “Se”, “en”, “ns”, “ti”, “iv”, “ve”, “ H”, “Ha”, “as”, “sh”, “hi”, “ f”, “fo”, “or”, “r ”, “ D”, “at”, “io”}

$$Sim(D1, D2) = \frac{|D1 \cap D2|}{|D1 \cup D2|} = \frac{34}{45} = 0.7556$$

Model Choice

There are different model choices which are based on the application context :

1. **White space?** Should we include spaces, and returns? Sometimes.plane has touch down versus threw a touchdown.
2. **Capitalization?** Hashing versus hashing. Can help distinguish proper nouns.
3. **Punctuation?** May be indication of education level, or dialects. For instance English is punctuated differently in US and India. Punctuation is used differently in new articles (very proper style), blogs(more informal), and twitter (what is punctuation?).
4. **Characters vs. Words?** Long enough shingles with characters can simulate words, but will have more false positives. Can pick up other dialect patterns. But is less interpretable.
5. **How large should k be?**
6. **Stopping Words?** Words like {a, you, for, the, to, and, that, it, is, ...} are very common, and called stop words. Sometimes omit these (typically in bag of words).

Characteristic Matrix of Document sets: For above example of 2-shingle(word level) the characteristic matrix is as follow:

$D1 \cup D2$	$D1$	$D2$
"De duplication"	1	1
"duplication Using"	1	0
"Using Locality"	1	1
"Locality Sensitive"	1	1
"Sensitive Hashing"	1	1
"Hashing for"	0	1
"for De"	0	1

For large collection of documents, the columns of the characteristic matrix become sparse and large, so its too difficult to handle document sets efficiently .
So our goal is to compute a "signature" for each document set, so that similar documents have similar signatures (and dissimilar docs are unlikely to have similar signatures).

Hashing

The idea of hashing is to convert each document to a small signature using a hashing function H. Suppose a document in our corpus is denoted by d. Then:

- H(d) is the signature and it's small enough to fit in memory
- If similarity(d1,d2) is high then Probability(H(d1)==H(d2)) is high
- If similarity(d1,d2) is low then Probability(H(d1)==H(d2)) is low

Choice of hashing function is tightly linked to the similarity metric we're using. For Jaccard similarity the appropriate hashing function is min-hashing.

Min-Hashing

Minhash(π) of a set is the number of the row (element) with first non-zero in the **permuted order π**.
Min-Hash Signature: Let {h1, h2, ..., hn} be different minhash functions (i.e. different permutations).
Then signature for set S is:

$$SIG(S) = [h1(S), h2(S), ..., hn(S)]$$

Let $S1$ and $S2$ are two sets then the jaccard similarity is $S1$ and $S2$ in terms of signature is:

SIM(S1,S2) $\approx$ ratio of equal elements of SIG(S1) and SIG(S2)

MinHasing Algorithm

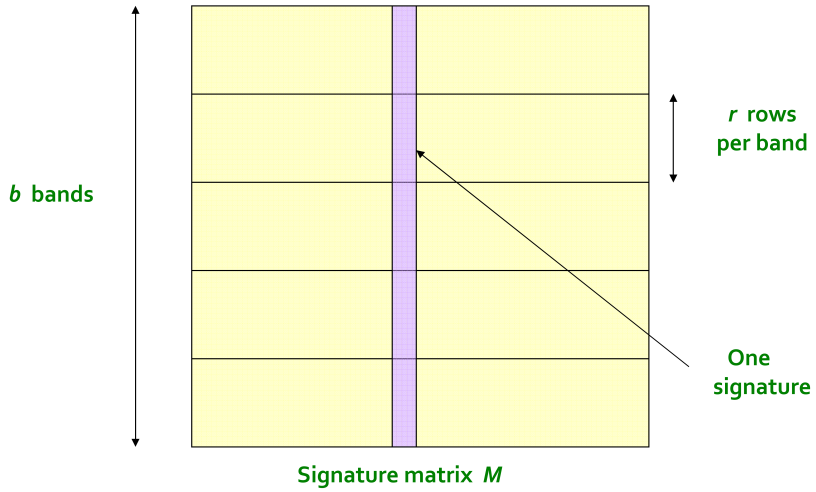
```
For each row r = 0, 1, ..., N-1 of the characteristic matrix:
1. Compute h1(r), h2(r), ..., hn(r)
2. For each column c:
   1. If column c has 0 in row r,  do nothing
   2. Otherwise, for each i = 1,2, ..., n set
                 SIG(i, c) to be min(hi(r), SIG(i, c))
```

Locality Sensitive Hashing

The basic idea of locality sensitive hashing to use a function f(x,y) that tells whether x and y is a candidate pair(i.e. a pair of elements whose similarity must be evaluated).

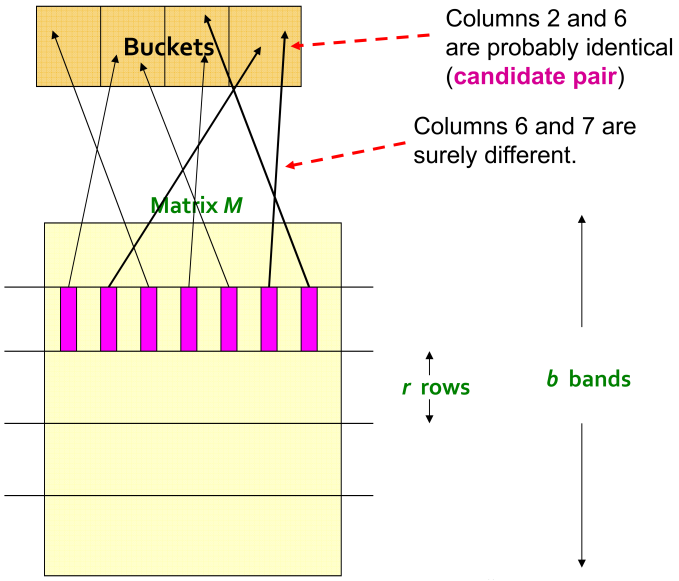
Method to find such f from Min-hash Signature matrix:

1. Divide the rows of signature matrix in b band having r rows.



Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

2. For each band hash the column in k buckets, buckets contains column pairs if there are identical in particular band and the pair of a bucket are called **candidate pairs**

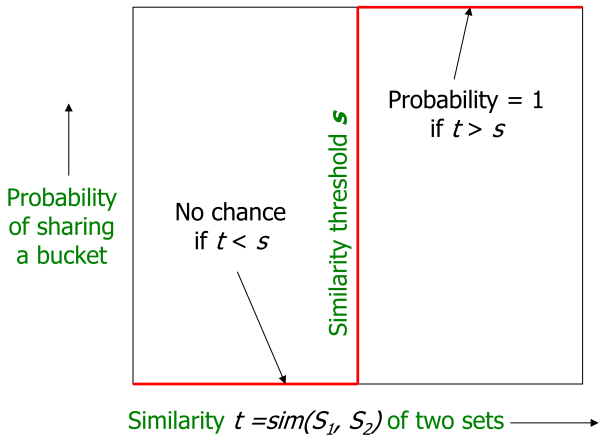


Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

3. Tune value of b and r such that similar columns goes to same bucket with high probability and dissimilar columns goes to same bucket with less probability.

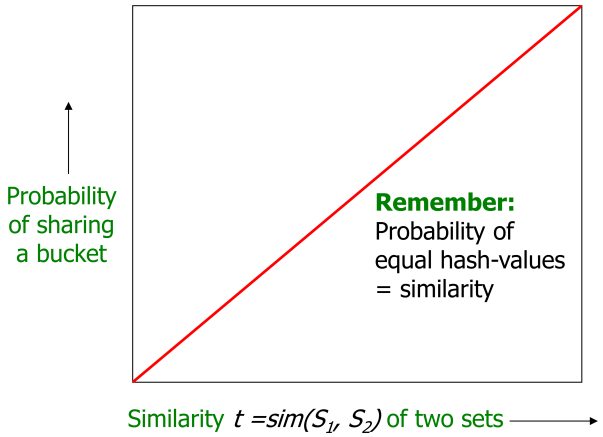
Interpretation of b bands for r rows:

- We want that for column whose similarity is greater than threshold t will goes to same bucket with probability 1 .



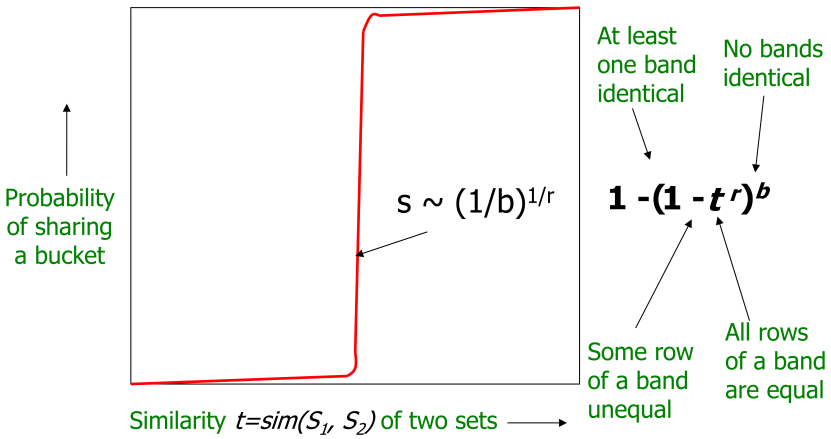
Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

- For $b = 1$ and $r = 1$:



Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

- For b bands and r rows:



Source: J. Leskovec, A. Rajaraman, J. Ullman: Mining of Massive Datasets

Experimental Study and Results

To check the Working and performance of the implemented locality sensitive hashing algorithm, **20 Newsgroup** text data set containing 600 hundred text files is taken in consideration. The actual truth about the data set is not known, so to find the actual truth first we computed the jaccard similarity of all combinations of text files, and those having jaccard similarity greater than 0.55, kept in one bucket/cluster representing the duplicates.We found 8 such clusters and are as follow:

Table 1: Actual Truth

Cluster Number	File ID's
1	**[51150, 51240]**
2	**[51270, 51307]**
3	**[53072, 53178]**
4	**[37917, 37950]**
5	**[38216, 38255]**
6	**[38227, 38330]**
7	**[38320, 38432]**
8	**[38438, 38442, 38458]**

LSH Parameters Values

Results for following experimental setting are as follow :

- k (for k-shingle) = 5
- Total number of hash functions for Min-hashing = 100
- Number of bands (b) = 10
- Number of rows in a band (r) = 6

For the above setting for LSH we got **11** clusters and are as follow:

Table 2: Predicted Duplicates

Cluster Number	File ID's
1	**[38227, 38330]**
2	**[37917, 37950]**
3	**[38354, 38366]**
4	**[38435, 38457]**
5	**[53117, 53134]**
6	**[53174, 53195]**
7	**[38438, 38442, 38458]**
8	**[51186, 51210]**
9	**[38396, 38461]**
10	**[53072, 53178]**

In above table the blue colour cluster are correct one i.e. having jaccard similarity greater than 0.55 and the red colour clusters are false positive i.e. clusters whose actual jaccard similarity is less 0.55 but predicted as duplicate by LSH.
Form table [**1**] and [**2**], we can see that LSH is able to cluster only 4 duplicates clusters correctly out of 8.

On repeating the same experiment for same above setting we got **15** clusters and are as follow:

Table 3: Predicted Duplicates

Cluster Number	File ID's
1	**[51150, 51240]**
2	**[53174, 53195]**
3	**[37941, 37960]**
4	**[38320, 38432]**
5	**[53072, 53178]**
6	**[38442, 38458]**
7	**[51197, 51198]**
8	**[53064, 38099]**
9	**[38217, 38330]**
10	**[51290, 51304]**
11	**[51197, 51258]**
12	**[38217, 38227]**
13	**[51194, 51277]**
14	**[38216, 38255]**
15	**[53173, 53191]**
16	**[38217, 38227, 38330]**

Form table [**1**] and [**3**], we can see that LSH is able to cluster only 4 duplicates out off which one is similar as correctly detected in talble[**2**]. Hence, we are able to get **7** duplicates out off **8** only in **2** runs of the **LSH** algorithm and only in **26** pair comparisons.

References

[1] A. Rajaraman and J. D. Ullman, *Mining of massive datasets*. Cambridge University Press, 2011.